

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа №1
по дисциплине: Операционные системы
по теме: Системные вызовы. Базовая работа с процессами в ОС Linux
(Ubuntu)

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: доц. Островский Алексей Мичеславович,
асс. Четвертухин Виктор Романович

Цель работы: Изучить основы работы с системными вызовами и процессами в операционной системе Linux (Ubuntu).

Вариант 1

Создать путем порождения процессов двоичное дерево из 7-ми вершин (процессов) со связями «родитель-потомок» путем последовательных вызовов функции `fork()`. В этом дереве каждый процесс (кроме листьев) должен порождать двух потомков. Превратить дерево в граф, путем замещения одного листа корнем. Корректно завершить все процессы. Осуществлять проверку программы путем мониторинга процессов через утилиты (`ps` или `top`)

Создание двоичного дерева с 7-мью вершинами

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
#include <sys/wait.h>
#include <signal.h>

typedef struct binary_tree {
    pid_t left_child;
    pid_t right_child;
} binary_tree;

int creating_binary_tree (int level, int max_level) {
    if (level == max_level) {
        return 0;
    }

    binary_tree bt;

    bt.left_child = fork();
    if (bt.left_child == 0) {
        printf("Был создан процесс [%d] для ЛЕВОГО потомка процесса [%d]\n",
            getpid(), getppid());
        creating_binary_tree(level + 1, max_level);
        exit(0);
    }

    bt.right_child = fork();
    if (bt.right_child == 0) {
        printf("Был создан процесс [%d] для ПРАВОГО потомка процесса [%d]\n",
            getpid(), getppid());
        creating_binary_tree(level + 1, max_level);
        exit(0);
    }

    // Ждем завершения обоих потомков
    wait(NULL);
    wait(NULL);
}

int main() {
    int level = 0;
    int max_level = 2;
    creating_binary_tree(level, max_level);

    // Ожидаем завершения всех процессов
    wait(NULL);
}
```

Превращение дерева в граф, путем замещения одного листа корнем.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
#include <sys/wait.h>
#include <signal.h>

typedef struct binary_tree {
    pid_t left_child;
    pid_t right_child;
} binary_tree;

int creating_binary_tree (int level, int max_level) {
    if (level == max_level) {
        return 0;
    }

    binary_tree bt;

    bt.left_child = fork();
    if (bt.left_child == 0) {
        printf("Был создан процесс [%d] для ЛЕВОГО потомка процесса [%d]\n",
            getpid(), getppid());
        creating_binary_tree(level + 1, max_level);
        exit(0);
    }

    bt.right_child = fork();
    if (bt.right_child == 0) {
        printf("Был создан процесс [%d] для ПРАВОГО потомка процесса [%d]\n",
            getpid(), getppid());
        creating_binary_tree(level + 1, max_level);
        exit(0);
    }

    // Ждем завершения обоих потомков
    wait(NULL);
    wait(NULL);
}

int main() {
    int level = 0;
    int max_level = 2;
    creating_binary_tree(level, max_level);
    if (fork() == 0) {
        printf("Лист становится новым корнем: PID %d\n", getpid());
        exit(0);
    }
    wait(NULL);
}
```

Вывод программы:

```
wwwxtus@DESKTOP-I1F97A5:~/Универ/Операционные системы$ gcc lab_OC_1.c -o lab_OC_1
wwwxtus@DESKTOP-I1F97A5:~/Универ/Операционные системы$ ./lab_OC_1
Был создан процесс [1370] для ЛЕВОГО потомка процесса [1369]
Был создан процесс [1371] для ПРАВОГО потомка процесса [1369]
Был создан процесс [1372] для ЛЕВОГО потомка процесса [1370]
Был создан процесс [1373] для ПРАВОГО потомка процесса [1370]
Был создан процесс [1374] для ЛЕВОГО потомка процесса [1371]
Был создан процесс [1375] для ПРАВОГО потомка процесса [1371]
Лист становится новым корнем: PID 1376
```

Добавлю sleep(), чтобы процессы сразу не завершались

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

typedef struct binary_tree {
    pid_t left_child;
    pid_t right_child;
} binary_tree;

int creating_binary_tree(int level, int max_level) {
    if (level == max_level) {
        return 0;
    }
    binary_tree bt;
    bt.left_child = fork();
    if (bt.left_child == 0) {
        printf("Был создан процесс [%d] для ЛЕВОГО потомка процесса [%d]\n",
            getpid(), getppid());
        sleep(10);
        creating_binary_tree(level + 1, max_level);
        exit(0);
    }

    bt.right_child = fork();
    if (bt.right_child == 0) {
        printf("Был создан процесс [%d] для ПРАВОГО потомка процесса [%d]\n",
            getpid(), getppid());
        sleep(10);
        creating_binary_tree(level + 1, max_level);
        exit(0);
    }
    wait(NULL);
    wait(NULL);
    return 0;
}

int main() {
    int level = 0;
    int max_level = 2;
    creating_binary_tree(level, max_level);
    if (fork() == 0) {
        printf("Лист становится новым корнем: PID %d\n", getpid());
        sleep(10);
        exit(0);
    }
    wait(NULL);
    return 0;
}
```

Отообразил созданные процессы с помощью команды htop

Main		I/O									
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
13524	wwwxtus	20	0	2548	1060	972	S	0.0	0.0	0:00.00	./lab_OC_1
13525	wwwxtus	20	0	2680	88	0	S	0.0	0.0	0:00.00	./lab_OC_1
13526	wwwxtus	20	0	2680	88	0	S	0.0	0.0	0:00.00	./lab_OC_1
13557	wwwxtus	20	0	2680	92	0	S	0.0	0.0	0:00.00	./lab_OC_1
13558	wwwxtus	20	0	2680	92	0	S	0.0	0.0	0:00.00	./lab_OC_1
13559	wwwxtus	20	0	2680	92	0	S	0.0	0.0	0:00.00	./lab_OC_1
13560	wwwxtus	20	0	2680	92	0	S	0.0	0.0	0:00.00	./lab_OC_1

Важные данные в колонках:

VIRT (Virtual memory size): Сколько виртуальной памяти занимает процесс

SHR (Shared memory size): Объём памяти, который процесс использует совместно с другими процессами

%CPU: Процент использования процессора этим процессом

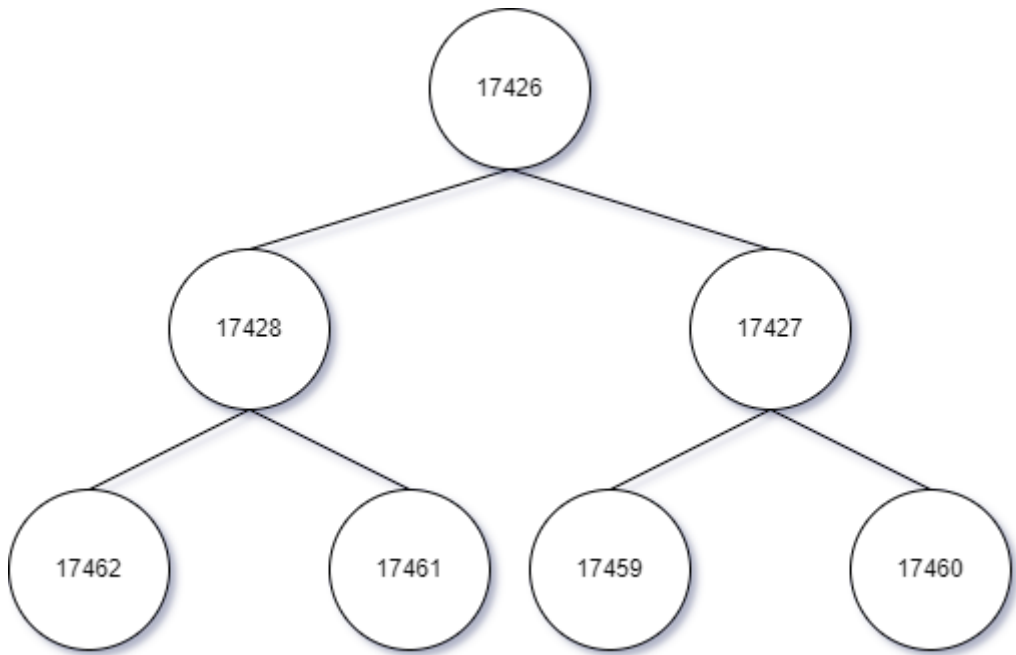
TIME+: Общее процессорное время, затраченное на выполнение процесса с момента его запуска

Command: Команда, с помощью которой был запущен процесс

Вывод в htop с помощью дерева как и нужно было по заданию

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
955	root	20	0	2492	120	0	S	0.0	0.0	0:00.00	/init
956	root	20	0	2492	128	0	S	0.0	0.0	0:00.22	└─ /init
957	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.73	└─ /home/wwwxtus/.vscode
958	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.00	└─ /home/wwwxtus/.vscode
959	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.00	└─ /home/wwwxtus/.vscode
960	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.01	└─ /home/wwwxtus/.vscode
961	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.02	└─ /home/wwwxtus/.vscode
962	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.01	└─ /home/wwwxtus/.vscode
963	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.00	└─ /home/wwwxtus/.vscode
11213	root	20	0	2480	120	0	S	0.0	0.0	0:00.00	└─ /init
11214	root	20	0	2496	124	0	S	0.0	0.0	0:00.01	└─ /init
11219	wwwxtus	20	0	6332	5552	3604	S	0.0	0.1	0:00.07	└─ -bash
17426	wwwxtus	20	0	2548	1096	1008	S	0.0	0.0	0:00.00	└─ ./lab_OC_1
17427	wwwxtus	20	0	2680	88	0	S	0.0	0.0	0:00.00	└─ ./lab_OC_1
17459	wwwxtus	20	0	2680	92	0	S	0.0	0.0	0:00.00	└─ ./lab_OC_1
17460	wwwxtus	20	0	2680	92	0	S	0.0	0.0	0:00.00	└─ ./lab_OC_1
17428	wwwxtus	20	0	2680	88	0	S	0.0	0.0	0:00.00	└─ ./lab_OC_1
17461	wwwxtus	20	0	2680	92	0	S	0.0	0.0	0:00.00	└─ ./lab_OC_1
17462	wwwxtus	20	0	2680	92	0	S	0.0	0.0	0:00.00	└─ ./lab_OC_1

Визуализирую в привычном виде исходя из того, что выдала команда htop



Дерево процессов после того как один из листьев сделал корнем

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
955	root	20	0	2492	120	0	S	0.0	0.0	0:00.00	/init
956	root	20	0	2492	128	0	S	0.0	0.0	0:00.22	└─ /init
957	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.73	└─ /init
958	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.00	└─ /home/wwwxtus/.vs
959	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.00	└─ /home/wwwxtus/
960	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.01	└─ /home/wwwxtus/
961	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.02	└─ /home/wwwxtus/
962	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.01	└─ /home/wwwxtus/
963	wwwxtus	20	0	967M	49968	38552	S	0.0	0.6	0:00.00	└─ /home/wwwxtus/
11213	root	20	0	2480	120	0	S	0.0	0.0	0:00.00	└─ /init
11214	root	20	0	2496	124	0	S	0.0	0.0	0:00.01	└─ /init
11219	wwwxtus	20	0	6332	5552	3604	S	0.0	0.1	0:00.07	└─ -bash
17426	wwwxtus	20	0	2548	1096	1008	S	0.0	0.0	0:00.00	└─ ./lab_OC_1
17493	wwwxtus	20	0	2680	88	0	S	0.0	0.0	0:00.00	└─ ./lab_OC_1

Визуализирую



Из-за того, что процесс правый уже завершился, после изменения листа на корень отображается только левый процесс

Вывод

В данной работе я построил бинарное дерево процессов с помощью создания дочерних процессов от родительских путём построения обычного бинарного дерева, а потом заменив лист новым корнем сделал из бинарного дерева граф, сложностей не заметил, узнал как можно создавать процессы и от них создавать дочерние процессы, узнал лучше возможности плагина htop и поработал с ним